Vision-Language Model Verifiers for GUI Action Validation: A Study on Training Strategies and Distillation

[Author Name] Affiliation>Affiliation

[email]

December 2025

Abstract

Autonomous GUI agents that interact with mobile and desktop interfaces require robust mechanisms to verify whether their executed actions successfully contribute to task completion. This research presents a comprehensive study on training Vision-Language Models (VLMs) as action verifiers, exploring multiple training paradigms including supervised fine-tuning, teacher-student distillation with GPT-5.2, and data corruption strategies for generating negative samples. We develop verifiers based on the Qwen3-VL (4B, 8B) and Qwen2.5-VL (3B, 7B) architectures and propose trajectory-aware corruption techniques for creating balanced training datasets. This document describes our methodology and implementation details, with comprehensive evaluation results to be added upon completion of extended testing.

1 Introduction

1.1 Motivation

The emergence of autonomous GUI agents represents a significant advancement in human-computer interaction research. These agents, powered by large language models and vision-language models, can interpret natural language instructions and execute corresponding actions on graphical user interfaces. From booking flights to managing emails, from online shopping to social media posting, these agents promise to automate complex multi-step tasks that traditionally required human intervention. However, a critical challenge remains largely unaddressed: determining whether an executed action actually made progress toward the intended goal.

The importance of this verification capability cannot be overstated. Without reliable action verification, autonomous agents operate in an open-loop manner, blindly executing actions without confirmation of success. This leads to cascading failures where a single undetected error propagates through subsequent actions, ultimately resulting in task failure. Consider a shopping agent tasked with finding the cheapest Nintendo Switch controller. The agent might tap on the search bar, type a query, and press enter. But how do we know the action succeeded? The search might have failed silently due to a network error. The wrong application might have opened because of an ambiguous tap location. The interface might have responded unexpectedly due to a popup or advertisement. Without verification, the agent would proceed to the next step, operating on incorrect assumptions about the current state.

The consequences of unverified actions extend beyond simple task failure. In high-stakes applications such as financial transactions, healthcare systems, or enterprise workflows, erroneous actions can have significant real-world implications. An agent that believes it has successfully completed a bank transfer when the transaction actually failed could lead to missed payments

and penalties. An agent managing calendar appointments might create conflicts or miss important meetings if it cannot verify that its scheduling actions succeeded. These scenarios highlight the fundamental need for robust action verification as a core capability of autonomous GUI agents.

1.2 The Verification Challenge

Action verification in GUI environments presents unique challenges that distinguish it from traditional computer vision tasks. Unlike object detection or image classification, verification requires understanding the semantic relationship between two states (before and after an action), the intended effect of an action, and whether the observed change aligns with the user's goal. This is fundamentally a multimodal reasoning task that combines visual understanding with natural language comprehension.

The visual component alone involves significant complexity. GUI screenshots contain rich hierarchical information including text, icons, buttons, input fields, and dynamic content. The verifier must parse this visual information, identify relevant UI elements, and track changes between states. Some changes are obvious—a new screen appearing, text being entered into a field—while others are subtle, such as a checkbox being selected or a slight color change indicating a toggle state. The verifier must be sensitive enough to detect meaningful changes while robust enough to ignore irrelevant visual differences such as timestamp updates or advertisement rotations.

The linguistic component adds another layer of complexity. Actions are described in natural language with varying levels of specificity. A tap action might be described as "tap the search button" or "tap at coordinates (512, 1240)" or simply "search." The subgoal might be phrased as "find controllers" or "search for Nintendo Switch accessories" or "locate the cheapest gaming peripheral." The verifier must understand these descriptions and evaluate whether the observed state transition represents meaningful progress toward the stated objective.

Perhaps most challenging is the need to reason about partial progress. Many subgoals require multiple actions to complete. Searching for a product involves tapping the search bar, entering text, and pressing search. Each individual action represents partial progress toward the subgoal. The verifier must distinguish between actions that make valid progress (even if they don't complete the subgoal) and actions that fail or are counterproductive. This requires understanding the logical structure of task execution and the role of each action within that structure.

1.3 Our Approach

This research addresses the verification challenge by training Vision-Language Models to serve as action verifiers. Given a BEFORE screenshot (the state before an action), an AFTER screenshot (the state after the action), and a description of the intended action and subgoal, the verifier must determine whether the action successfully contributed to progress. This binary classification task—answering "Yes" or "No" with supporting reasoning—is fundamental to building reliable autonomous agents.

We explore multiple training paradigms to develop effective verifiers. Our first approach employs supervised fine-tuning on a curated dataset of labeled action verification examples. We apply Low-Rank Adaptation (LoRA) to enable efficient training of large vision-language models on consumer hardware. The second approach introduces teacher-student distillation, using GPT-5.2 to generate high-quality reasoning chains that improve the student model's explanatory capabilities. The third approach addresses the challenge of dataset imbalance through trajectory-aware data corruption, generating realistic negative samples that simulate various failure modes.

We implement these approaches on two families of vision-language models from the Qwen series: Qwen3-VL (4B and 8B parameters) and Qwen2.5-VL (3B and 7B parameters). This selec-

tion enables systematic study of how model capacity and architecture affect verification performance. By training and evaluating multiple model variants across different training paradigms, we aim to provide insights into the trade-offs between model size, training approach, and verification accuracy.

1.4 Contributions

Our contributions are threefold. First, we develop and compare training strategies for VLM verifiers, including supervised fine-tuning with LoRA and teacher-student distillation using GPT-5.2 generated reasoning. We demonstrate that distillation can improve the quality and consistency of verification explanations while maintaining classification accuracy. Second, we introduce trajectory-aware data corruption techniques that generate realistic negative samples to address dataset imbalance. Unlike random corruption, our approach simulates specific failure modes that occur in real agent deployments, including cross-task confusion, step skipping, and action ineffectiveness. Third, we implement a complete training pipeline with environment setup, data processing, training scripts, and evaluation tools that enables systematic experimentation with different model architectures and training approaches. This pipeline is designed for reproducibility and extensibility, allowing other researchers to build upon our work.

2 Problem Formulation

The action verification task takes as input a tuple consisting of a BEFORE screenshot, an AFTER screenshot, an action description, and a current subgoal. The verifier must produce a binary verdict indicating whether the action succeeded, along with a natural language explanation supporting the verdict.

The verification decision requires considering multiple factors simultaneously. The verifier must understand the visual content of both screenshots, recognizing UI elements and their states. It must comprehend the semantic meaning of the action description and the subgoal. Finally, it must evaluate whether the observed transition—the change from BEFORE to AFTER—aligns with what the action was supposed to accomplish given the stated objective.

In practice, the input is structured as a conversation between a user providing the context and screenshots, and an assistant providing the verification response. The user message includes the global task description, current subgoal, action taken, and both BEFORE and AFTER images. The assistant response is a JSON object containing an answer field ("Yes" or "No") and a reason field explaining the decision.

The challenge is compounded by the diversity of possible failure modes. Actions can fail completely, where nothing visible happens despite the action being executed. They can fail partially, showing some change but not the intended outcome. They can succeed at the wrong task, such as opening the correct type of element but in the wrong location. They can appear to succeed while actually failing, showing expected visual changes that mask an underlying error. A robust verifier must distinguish all these cases from genuine success.

The formal objective is to train a model that maximizes the probability of correct classification across all samples in the training distribution. However, beyond raw accuracy, we also care about the quality of the reasoning provided. A verifier that gives correct answers with poor explanations is less useful than one that provides coherent reasoning that can be inspected and trusted by downstream systems or human operators.

3 Dataset

3.1 Data Source and Collection

The training data is derived from mobile device interaction recordings capturing GUI-based task execution. Each recording documents a user or agent performing a multi-step task on a mobile device, with screenshots captured before and after each action. The dataset covers diverse task categories including web shopping on platforms like Amazon, Walmart, and eBay; information management tasks involving calendar, notes, and settings applications; social sharing on platforms like Twitter, Instagram, and TikTok; media entertainment including music and video applications; and general tool usage such as browsers, maps, and utility applications.

The diversity of tasks and applications is essential for training verifiers that generalize beyond specific interfaces. A verifier trained only on shopping tasks might fail when presented with calendar management or social media posting. By including examples from multiple domains, we aim to develop verifiers that understand general principles of GUI interaction rather than memorizing specific interface patterns.

3.2 Data Format

Each sample in the dataset consists of several components. The BEFORE screenshot captures the device state immediately before an action is executed, typically at 1080 by 2400 pixel resolution. The AFTER screenshot captures the state after the action completes. The global task description provides the overall user objective in natural language, such as "Find the cheapest Nintendo Switch controller on Walmart." The current subgoal specifies the immediate objective being worked toward, such as "Search for the product." The action description includes the action type (Tap, Swipe, Type, or combined actions) along with relevant parameters such as coordinates and text input. The ground truth label indicates whether the action succeeded (Yes) or failed (No). Finally, the reasoning explanation provides a natural language justification for the label.

3.3 Dataset Statistics

The filtered dataset used for training contains 648 training samples and 71 validation samples. Each sample includes two images (BEFORE and AFTER), resulting in 1,436 total images. The dataset spans approximately 50 unique tasks across the categories described above.

The class distribution exhibits significant imbalance, with 517 positive samples (79.8%) and only 131 negative samples (20.2%). This imbalance reflects the natural distribution where most deliberate actions in recorded trajectories succeed—after all, the recordings capture users or agents attempting to complete tasks, and most attempts succeed. However, this imbalance poses challenges for training, as models may learn to simply predict "Yes" for all inputs. We address this through data corruption techniques described in the methodology section.

3.4 Data Preprocessing

The raw data underwent several preprocessing steps before training. Samples with missing or corrupted images were identified and removed to ensure training stability. All samples were converted to a consistent JSONL format with standardized field names and structures. Image paths were verified and normalized to enable reliable loading during training. Samples with incomplete metadata, such as missing action descriptions or subgoals, were excluded.

After filtering, the dataset was split into training and validation sets with a 90/10 ratio. We ensured no task overlap between splits to evaluate generalization—the validation set contains tasks not seen during training, providing a more realistic assessment of model performance on novel scenarios.

4 Methodology

4.1 Supervised Fine-Tuning

Our first training approach employs supervised fine-tuning on the curated dataset of labeled action verification examples. We utilize the Unsloth framework for efficient fine-tuning of vision-language models, which provides optimized implementations that reduce memory requirements and training time. To further improve efficiency, we apply Low-Rank Adaptation (LoRA), which decomposes weight updates into low-rank matrices. This dramatically reduces the number of trainable parameters from millions to thousands while maintaining model quality.

The training data consists of input-output pairs where the input includes the screenshots and text context, and the output is the expected verification response in JSON format. The model learns to generate appropriate responses by minimizing the cross-entropy loss between its predictions and the ground truth labels. We configure LoRA with rank 16 and scaling factor 16, targeting the attention and feed-forward projection matrices. Training proceeds for 3 epochs with a batch size of 1 and gradient accumulation over 8 steps, yielding an effective batch size of 8. We use a learning rate of 2e-4 with cosine annealing and 10% warmup.

A key advantage of supervised fine-tuning is its simplicity and reliability. The training objective is straightforward, the optimization is stable, and results are reproducible. However, we observed that the quality of reasoning in the original labels varied considerably. Some labels provided detailed explanations of why an action succeeded or failed, while others offered only brief justifications like "Action completed" or "Nothing changed." This inconsistency motivated our exploration of distillation as a method to improve reasoning quality.

4.2 Teacher-Student Distillation

To address the variability in reasoning quality, we implemented a teacher-student distillation approach using GPT-5.2 as the teacher model. GPT-5.2 is OpenAI's latest flagship model with enhanced vision capabilities and strong reasoning abilities. The intuition behind distillation is that by training on high-quality, consistent reasoning generated by a capable teacher, the student model can learn to produce similar reasoning even though it has far fewer parameters.

The distillation process proceeds in two stages. In the first stage, we process each training sample through GPT-5.2 Vision, providing the BEFORE and AFTER screenshots along with the task context. We augment the original prompt with explicit instructions for chain-of-thought reasoning, asking GPT-5.2 to describe what it sees in the BEFORE image, describe what it sees in the AFTER image, explain what changed between the two states, evaluate whether the change aligns with the subgoal, and provide a final verdict with clear reasoning.

This structured approach yields detailed, consistent responses that follow a predictable format. The responses are typically longer and more thorough than the original human labels, providing step-by-step reasoning that can be followed and verified. We processed all 648 training samples through GPT-5.2, generating a new dataset of enhanced labels.

In the second stage, we continue training the supervised fine-tuned model on these GPT-5.2 generated responses. We use a lower learning rate of 1e-4 to preserve the classification ability learned during supervised fine-tuning while adapting to the more detailed reasoning style. The maximum sequence length is increased to 8192 tokens to accommodate the longer explanations.

The total cost for GPT-5.2 distillation was approximately 7-8 USD for processing all samples, representing a reasonable one-time investment for improved reasoning quality. This cost-effectiveness makes distillation an attractive option for researchers with limited budgets.

4.3 Data Corruption for Negative Sample Generation

The significant class imbalance in our dataset poses challenges for training robust verifiers. With 80% positive samples, a model could achieve high accuracy by simply predicting "Yes"

for all inputs, while completely failing to detect actual failures. To address this, we developed trajectory-aware data corruption techniques that generate realistic negative samples.

Our corruption strategy is inspired by the observation that GUI interactions form trajectories: sequences of actions that progress toward a goal. By corrupting individual transitions—replacing the correct AFTER state with an incorrect one—we create examples of what failure looks like. Importantly, we design corruptions that simulate specific failure modes rather than random noise.

We implement four types of corruption, each applied with equal probability. Cross-task corruption replaces the AFTER image with one from a completely different task, simulating scenarios where the wrong application opens or an unexpected screen appears. Wrong-step-future corruption uses an AFTER image from a later step in the same task, simulating scenarios where the agent skips ahead or the interface changes unexpectedly. Wrong-step-past corruption uses an AFTER image from an earlier step, simulating scenarios where progress is lost or the interface reverts. No-change corruption sets the AFTER image equal to the BEFORE image, simulating scenarios where the action has no effect.

Each corruption type generates samples with appropriate rejection reasons. Cross-task corruptions receive reasons like "The screen shows a completely different application than expected." No-change corruptions receive reasons like "The expected change did not occur; the screen remains unchanged." These generated reasons provide training signal for the model to learn appropriate explanations for different types of failures.

By generating corrupted samples from positive examples, we can balance the dataset to achieve a 50/50 ratio of positive and negative samples. This balanced training data helps the model learn to distinguish success from failure rather than defaulting to the majority class.

4.4 Direct Preference Optimization

We also explored Direct Preference Optimization (DPO) as an alternative training approach. DPO enables training on preference pairs—examples showing which of two responses is preferred—without requiring reinforcement learning. For action verification, we construct pairs where the preferred response correctly identifies a genuine success, while the dispreferred response incorrectly accepts a corrupted failure.

However, our experiments with DPO on vision-language models encountered significant technical challenges. The multimodal nature of the data created issues with image serialization during the multiprocessing required for efficient training. Memory requirements were higher than expected because DPO requires processing both chosen and rejected examples simultaneously. Training was substantially slower than standard supervised fine-tuning.

While we have implemented the DPO infrastructure and generated preference pairs, we consider this approach experimental pending resolution of these technical issues. The fundamental idea remains promising—teaching models to prefer correct verifications over incorrect ones—but practical implementation for multimodal models requires further development.

5 Implementation

5.1 Model Architectures

We implement our training approaches on two families of vision-language models from the Qwen series. The Qwen3-VL family represents the latest generation, available in 4B and 8B parameter variants. These models feature improved vision-language alignment and enhanced instruction-following capabilities. The Qwen2.5-VL family represents the previous generation, available in 3B and 7B parameter variants. While slightly older, these models remain highly capable and provide an interesting comparison point for studying architectural evolution.

All models are loaded with 4-bit quantization, which reduces memory requirements by approximately 4x compared to full precision. Fine-tuning uses LoRA adapters rather than updating all model weights, further reducing memory and enabling training on a single GPU with 24GB or more of VRAM. After training, the LoRA adapters are merged with the base model weights for efficient inference.

5.2 Training Pipeline

The complete training pipeline consists of several stages designed for reproducibility and ease of use. Environment setup installs all required dependencies including PyTorch, Unsloth, TRL, Transformers, and Pillow. It also applies necessary patches to the Unsloth library for proper vision-language model support. Data preparation filters and formats the training data, verifying that all images are accessible and samples are properly structured. Supervised fine-tuning trains the base model on labeled verification examples. Optional distillation generates GPT-5.2 responses and continues training on these enhanced labels. Optional data corruption generates balanced datasets for additional training. Finally, evaluation tests trained models on held-out validation data.

Each stage is implemented as a standalone Python script with command-line arguments for configuration. This modular design enables experimentation with different combinations of training approaches. Researchers can easily substitute different base models, adjust hyperparameters, or modify data processing without affecting other components.

5.3 Environment Requirements

Training requires several patches to the Unsloth library for proper VLM support. These patches address issues including missing imports in the compiler module, incorrect VLM detection in the RL module, and data collator conflicts in the dataset utilities. Our setup script automatically applies these patches along with installing all required dependencies.

The minimum hardware requirement is a GPU with 24GB of VRAM, such as an NVIDIA RTX 3090, RTX 4090, or A100. Training the 3B and 4B models is comfortable on 24GB, while the 7B and 8B models benefit from additional memory or gradient checkpointing. Training time varies from approximately 30 minutes for smaller models to several hours for larger ones, depending on hardware.

6 Experimental Design

6.1 Training Configurations

All models are trained using consistent hyperparameters to enable fair comparison. The LoRA configuration uses rank 16 and alpha 16, targeting all attention and feed-forward projection matrices. This configuration balances parameter efficiency with model expressiveness based on prior work showing its effectiveness for instruction-following tasks.

Training proceeds for 3 epochs, which we found sufficient for convergence without overfitting. The batch size is 1 with gradient accumulation over 8 steps, yielding an effective batch size of 8. This configuration accommodates memory constraints while providing stable gradient estimates. The learning rate is 2e-4 for supervised fine-tuning and 1e-4 for distillation, with cosine scheduling and 10% warmup. All training uses bfloat16 precision for computational efficiency.

6.2 Models Trained

We have trained the following model variants, enabling comparison across model sizes, architectures, and training approaches.

From the Qwen3-VL family, we trained Qwen3-VL-4B with supervised fine-tuning, Qwen3-VL-4B with distillation (continuing from the SFT model), Qwen3-VL-8B with supervised fine-tuning, and Qwen3-VL-8B with distillation. From the Qwen2.5-VL family, we trained Qwen2.5-VL-3B with supervised fine-tuning, Qwen2.5-VL-3B with distillation, Qwen2.5-VL-7B with supervised fine-tuning, and Qwen2.5-VL-7B with distillation.

This collection of eight models enables several analyses: comparison between 3B/4B and 7B/8B models to understand the impact of scale; comparison between Qwen2.5 and Qwen3 architectures; and comparison between SFT-only and distilled models to evaluate the impact of teacher-student learning.

6.3 Evaluation Protocol

Models are evaluated on held-out validation data using a standardized protocol. For each sample, we provide the BEFORE and AFTER images along with the task context, generate the model's response using greedy decoding to ensure reproducibility, and extract the predicted verdict from the JSON output. The primary metric is accuracy, defined as the percentage of samples where the predicted verdict matches the ground truth.

Additional metrics include average output token count, which measures response verbosity and correlates with reasoning detail; per-class accuracy, which measures performance on positive versus negative samples separately to detect bias toward the majority class; and qualitative analysis of reasoning quality, which assesses whether explanations are coherent, relevant, and helpful.

Comprehensive evaluation on extended test sets is planned. The current validation set of 71 samples provides initial insights, but more extensive testing across diverse scenarios is needed to fully characterize model performance and generalization capabilities.

7 Discussion

7.1 Design Decisions

Several key design decisions shape our approach and merit discussion. The choice of LoRA over full fine-tuning reflects practical constraints common to many research settings. Training large vision-language models with full parameter updates requires multiple high-end GPUs and substantial time. LoRA enables comparable results with a fraction of the computational resources, democratizing access to VLM fine-tuning for researchers without extensive infrastructure.

The distillation approach using GPT-5.2 as teacher was motivated by the observation that human labels varied significantly in reasoning quality. By training on GPT-5.2's structured chain-of-thought responses, we aim to improve not just classification accuracy but also the interpretability and trustworthiness of model outputs. In deployment scenarios, operators may want to inspect verification reasoning to build confidence in agent behavior. Consistent, well-structured explanations support this use case better than brief, variable labels.

The trajectory-aware corruption strategy was designed to generate negative examples that resemble real failure modes. Unlike random image swapping, which might produce obviously incorrect examples, our corruption types simulate specific scenarios: wrong applications opening (cross-task), steps being skipped or reversed (wrong-step variants), and actions having no effect (no-change). Training on these realistic failures should improve model robustness compared to training on artificial corruptions.

7.2 Trade-offs

We anticipate several trade-offs that our evaluation will quantify. There is likely a trade-off between model size and computational cost. Larger models (7B/8B) may achieve higher accu-

racy but require more resources for both training and inference. For deployment in resource-constrained settings, smaller models may be preferable even if accuracy is slightly lower.

Distillation may trade some classification accuracy for improved reasoning quality. The longer, more detailed outputs from distilled models require more inference time and may occasionally contain reasoning errors that simpler responses would avoid. However, the improved interpretability may be worth this cost in applications where explanation quality matters.

Training on balanced data may reduce false positives (incorrectly accepting failures) at the cost of some reduction in true positive rate (correctly accepting successes). The optimal balance depends on the deployment context. In high-stakes applications where failures are costly, accepting more false negatives to reduce false positives may be appropriate.

7.3 Limitations

Several limitations of our work should be acknowledged. The training data comes from mobile device recordings and may not generalize well to desktop interfaces, web applications, or different mobile platforms. The visual patterns, interaction paradigms, and failure modes differ across these contexts. Extending our approach to these domains would require collecting and annotating additional training data.

The corruption strategy, while trajectory-aware, generates synthetic negatives that may not capture all real failure modes. Some failures are subtle—an action might appear to succeed visually while failing at a deeper level, such as a form submission that shows success but doesn't actually save data. Detecting such failures would require additional signal beyond visual comparison.

The DPO approach remains experimental due to technical challenges with multimodal training. Resolving these issues could unlock additional improvements, but requires further engineering effort beyond the scope of current work.

8 Conclusion

This research presents a comprehensive methodology for training vision-language models as GUI action verifiers. We have developed and implemented multiple training approaches including supervised fine-tuning with LoRA, teacher-student distillation using GPT-5.2, and trajectory-aware data corruption for generating balanced training datasets. The complete pipeline includes environment setup, data processing, training scripts, and evaluation tools, all designed for reproducibility and extensibility.

We have trained verifiers based on the Qwen3-VL (4B, 8B) and Qwen2.5-VL (3B, 7B) architectures, creating a collection of models that enable systematic comparison across model sizes, architectures, and training approaches. Initial results are promising, and comprehensive evaluation on extended test sets will provide deeper insights into the trade-offs and capabilities of these verifiers.

The verification capability developed here addresses a critical gap in autonomous GUI agent systems. By accurately determining whether actions succeed, agents can detect errors in real-time, plan recovery strategies when things go wrong, and confirm task completion with confidence. These capabilities are essential for deploying GUI agents in real-world applications where reliability matters. As autonomous agents become more prevalent in our digital lives, robust verification will be increasingly important for ensuring they behave as intended.

References

1. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. Alibaba Group, 2023.

2. Hu, E. J., et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.

3. Rafailov, R., et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS 2023.

4. Ouyang, L., et al. Training Language Models to Follow Instructions with Human Feedback. NeurIPS 2022.

5. Unsloth: Efficient Fine-tuning for Large Language Models. `https://github.com/unslothai/unsloth`

A Environment Setup

To replicate this research, clone the repository and execute the setup_environment.sh script, which installs all required dependencies and applies necessary patches to the Unsloth library. The patches address three specific issues: missing psutil imports in the compiler module, incorrect VLM detection in the RL module, and data collator override in the dataset utilities. Without these patches, VLM training will fail with various cryptic errors.

After setup, verify the installation by importing the FastModel class from Unsloth. If the import succeeds without errors, the environment is correctly configured. Model training can then proceed using the provided scripts in the SFT and Teacher_Student_Distill directories.

B Code Repository

The complete code for this research is available at: `https://github.com/pockyitachi/Q-Verifier.git`

The repository is organized into several directories. The SFT directory contains the supervised fine-tuning code, including the main training script (train_verifier.py), evaluation script (test_verifier.py), and training data. The Teacher_Student_Distill directory contains the distillation code, including the GPT-5.2 response generation script and distillation training script. The Data_Corruption directory contains the trajectory-aware corruption code for generating balanced datasets. The DPO directory contains experimental code for Direct Preference Optimization.

C Prompt Examples

The base verification prompt structures the task as a conversation between a human user and an AI assistant. The user provides the global task description, current subgoal, action details including type and parameters, and BEFORE/AFTER screenshots. The assistant is instructed to act as a "mobile GUI reflector" that determines whether the state transition is valid.

For GPT-5.2 distillation, we augment this base prompt with explicit instructions for chain-of-thought reasoning. We clarify that a subgoal may require multiple actions to complete, and the evaluation should focus on whether this single action makes valid progress, not whether it fully completes the subgoal. We provide specific criteria for validity: an action is valid if it makes progress toward the subgoal, is a necessary preparatory step, or contributes to eventually achieving the objective. We then request a detailed five-step analysis covering the BEFORE state, AFTER state, changes observed, evaluation of validity, and final answer with reasoning.

The expected output format is a JSON object with two fields: "answer" containing either "Yes" or "No", and "reason" containing the explanation. SFT models typically produce concise reasons of one or two sentences, while distilled models produce longer, structured reasons following the chain-of-thought format.

D Results (To Be Completed)

This section will be populated with comprehensive evaluation results upon completion of extended testing. Planned analyses include performance comparison across all eight model variants, analysis of how accuracy scales with model size, comparison of SFT versus distilled models in terms of both accuracy and reasoning quality, evaluation of models trained on balanced versus imbalanced data, per-class accuracy breakdown to assess bias toward positive or negative predictions, and qualitative analysis of reasoning coherence and relevance.